A
PROJECT REPORT ON

URBAN HEAT STRESS HOTSPOT DETECTION AND COOLING STRATEGY OPTIMIZATION

SUBMITTED TO DR. BABASAHEB AMBEDKAR TECHNOLOGICAL UNIVERSITY

IN PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE OF

BACHELOR OF TECHNOLOGY
(COMPUTER SCIENCE AND ENGINEERING)

SUBMITTED BY

[Student Name 1]	[Exam No. 1]
[Student Name 2]	[Exam No. 2]
[Student Name 3]	[Exam No. 3]
[Student Name 4]	[Exam No. 4]



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSMSS, CHHATRAPATI SHAHU COLLEGE OF ENGINEERING

KANCHANWADI, PAITHAN ROAD, CHHATRAPATI SAMBHAJINAGAR
(AURANGABAD) – 431 002

DR. BABASAHEB AMBEDKAR TECHNOLOGICAL UNIVERSITY, LONERE

2026–2027

CERTIFICATE

This is to certify that the project report entitled

URBAN HEAT STRESS HOTSPOT DETECTION AND COOLING STRATEGY OPTIMIZATION

submitted by the following students is a bonafide record of the project work carried out under the supervision of Dr. G. J. Sahani. The work is approved for the partial fulfillment of the requirements of Dr. Babasaheb Ambedkar Technological University for the award of the degree of Bachelor of Technology in Computer Science and Engineering.

[Student Name 1]	[Exam No. 1]
[Student Name 2]	[Exam No. 2]
[Student Name 3]	[Exam No. 3]
[Student Name 4]	[Exam No. 4]

The submitted report shall be completed with the final student details, examination numbers, signatures, and institutional approval before formal submission.

Supervisor and head	Principal
(Dr. G. J. Sahani) Project Guide Computer Science and Engineering (Dr. S. P. Abhang) Head, Computer Science and Engineering	(Dr. G. B. Dongare) Principal CSMSS, Chhatrapati Shahu College of Engineering Chhatrapati Sambhajinagar

Place: Chhatrapati Sambhajinagar
Date: _____

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, Dr. G. J. Sahani, for the guidance, constructive review, and encouragement extended during the development of this project. The emphasis on understanding the problem, validating the results, and communicating the limitations of a technical system has been valuable throughout the work.

We thank Dr. S. P. Abhang, Head of the Department of Computer Science and Engineering, and Dr. G. B. Dongare, Principal of CSMSS, Chhatrapati Shahu College of Engineering, for the academic environment and institutional support that make project-based learning possible. We also acknowledge the faculty members and reviewers who contributed suggestions during the project development process.

This report documents the UrbanHeat AI repository and its geospatial machine-learning workflow. We acknowledge Team AstroByte—Srushti Bawaskar, Priyal Deshmukh, and Rishika Deshmukh—for the project contributions identified in the repository, including geospatial data preparation, machine learning, research, documentation, and dashboard development. These project credits are distinct from the student submission details to be completed on the title page.

The developers of Python, Pandas, NumPy, scikit-learn, XGBoost, SHAP, Streamlit, Plotly, Folium, and FastAPI have made robust open-source tools available for research and application development. We also acknowledge the Earth-observation and population-data providers whose datasets support the analytical workflow described in this report.

Finally, we thank our classmates, friends, and families for their patience and support. Feedback from early demonstrations helped improve the explanation of heat hotspots, population exposure, model uncertainty, and the responsible use of cooling-scenario estimates.

Student submission details and signatures should be inserted here before final institutional submission.

ABSTRACT

Urban heat exposure is spatially uneven: the hottest surface is not necessarily where the greatest number of people is exposed. This project, Urban Heat Stress Hotspot Detection and Cooling Strategy Optimization, develops a geospatial decision-support workflow for Chhatrapati Sambhajinagar (Aurangabad), Maharashtra. It combines satellite-derived environmental indicators, population exposure, explainable machine learning, spatial validation, and interactive scenario analysis.

The final Dataset V3 contains 9,893 sampled locations. Its predictors comprise the Normalized Difference Vegetation Index (NDVI), Normalized Difference Built-up Index (NDBI), elevation, population, and seven one-hot land-cover classes. Land surface temperature (LST) is the regression target. Latitude and longitude are retained for mapping and spatial grouping but excluded from model inputs. Linear Regression, Random Forest, and XGBoost are compared using five-fold GroupKFold validation over 334 geographic blocks. The saved V3 comparison reports a mean-fold R^2 of 0.5691, mean absolute error of 1.6645 °C, and root mean squared error of 2.1163 °C for XGBoost. An independent reproduction during report preparation produced closely aligned results.

The analytical products include observed-LST quartile hotspots, SHAP feature attribution, a population-aware Heat Vulnerability Index, and two notebook-based cooling scenarios. Urban greening produces a mean predicted reduction of 0.257 °C, while cool/permeable surfaces produce 0.076 °C across all sampled locations. Negative local effects are retained rather than hidden. A separate interactive simulator combines five intervention controls through transparent sensitivity assumptions shared by the Streamlit dashboard and FastAPI service. Its results are comparative planning estimates, not empirically validated causal forecasts.

The system provides hotspot and vulnerability views, model summaries, heat-driver analysis, priority tables, maps, and CSV exports. Repository checks confirm data integrity, feature-order consistency, grouped validation separation, scenario bounds, and agreement between API and shared inference. The framework supports transparent prioritisation, but implementation still requires field measurements, engineering assessment, and local feasibility review.

Keywords: urban heat; LST; XGBoost; spatial validation; SHAP; vulnerability; cooling.

TABLE OF CONTENTS

Title of chapter / section	Page No.
Preliminary sections	
Certificate / Acknowledgement / Abstract	i / ii / iii
List of abbreviations / figures / tables	vi / vii / viii
CHAPTER 1: INTRODUCTION	1
1.1 Project Introduction	1
1.2 Background and Motivation	2
1.5 Project Objectives	3
1.7 Existing System and Identified Gaps	4
CHAPTER 2: TECHNOLOGY STACK	5
2.1 Technology-Stack Overview	5
2.2 Python and Data-Processing Libraries	6
2.4 Earth-Observation and Geospatial Technologies	7
2.6 Application, API, and Deployment Technologies	8
CHAPTER 3: SYSTEM ANALYSIS AND REQUIREMENTS	9
3.1 Stakeholders and Functional Requirements	9
3.2 Non-Functional Requirements	10
3.3 Feasibility Analysis	11
3.5 Constraints, Assumptions, and Risks	12
CHAPTER 4: SYSTEM DESIGN AND ARCHITECTURE	13
4.1 Overall System Architecture	13
4.2 Context-Level Data Flow Diagram	14
4.4 Level 1 Processes and Data Stores	15
4.5 Cooling-Simulation Flowchart	16
4.6 Use-Case Design	17
4.7 Sequence and Module Design	18
CHAPTER 5: DATA AND MACHINE-LEARNING IMPLEMENTATION	19
5.1 Dataset Versions and Source Provenance	19
5.2 Data Audit, Cleaning, and Exploratory Analysis	20
5.3 Feature Engineering and Leakage Control	21
5.4 Model Development and Spatial Validation	22
5.5 Final XGBoost Configuration and Model Selection	23
5.6 Notebook Workflow and Reproducibility	24

TABLE OF CONTENTS (CONTINUED)

Title of chapter / section	Page No.
CHAPTER 6: DASHBOARD AND DECISION-SUPPORT IMPLEMENTATION	25
6.1 Dashboard Structure and Navigation	25
6.2 Hotspot and Vulnerability Views	26
6.3 Explainability and Model-Validation Views	27
6.4 Interactive Cooling-Strategy Simulator	28
6.5 Simulator Results and Export Behaviour	29
CHAPTER 7: DATA AND API DESIGN	30
7.1 Logical Data Model and File Contracts	30
7.2 REST API Design	31
7.4 Deployment, Security, and Integration Boundaries	32
CHAPTER 8: DASHBOARD ANALYTICS AND RESULTS	33
8.1 Heat and Vulnerability Analytics	33
8.2 Heat-Driver and Explainability Analytics	34
8.3 Cooling-Scenario and Priority Analytics	35
CHAPTER 9: TESTING AND VALIDATION	36
9.1 Testing Strategy and Data Integrity	36
9.2 API, Integration, and Functional Testing	37
9.4 Independent Spatial-Validation Reproduction	38
CHAPTER 10: RESULTS AND DISCUSSION	39
10.1 Achievement of Project Objectives	39
10.3 Limitations and Threats to Validity	40
CHAPTER 11: CONCLUSION AND FUTURE SCOPE	41
11.1 Advantages of the Developed System	41
11.3 Future Scope	42
APPENDIX A: FEASIBILITY AND COMPLEXITY ASSESSMENT	43
APPENDIX B: PAPER PUBLICATION / PRESENTATION	44
APPENDIX C: PLAGIARISM REPORT	45
REFERENCES	46

LIST OF ABBREVIATIONS

Abbreviation	Expansion
AI	Artificial Intelligence
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
CSV	Comma-Separated Values
CSCOE	Chhatrapati Shahu College of Engineering
CV	Cross-Validation
DBATU	Dr. Babasaheb Ambedkar Technological University
DFD	Data Flow Diagram
ESA	European Space Agency
GIS	Geographic Information System
HVI	Heat Vulnerability Index
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
LST	Land Surface Temperature
MAE	Mean Absolute Error
ML	Machine Learning
NDBI	Normalized Difference Built-up Index
NDVI	Normalized Difference Vegetation Index
NIR	Near Infrared
OOF	Out-of-Fold
QGIS	Quantum GIS
REST	Representational State Transfer
RMSE	Root Mean Squared Error
SHAP	SHapley Additive exPlanations
SRTM	Shuttle Radar Topography Mission
SR	Surface Reflectance
SWIR	Shortwave Infrared
UHI	Urban Heat Island
URL	Uniform Resource Locator
XGBoost	Extreme Gradient Boosting

LIST OF FIGURES

Number	Figure title	Page No.
1.1	Decision-support workflow from environmental inputs to planning outputs	1
2.1	Layered technology stack used by the project	5
3.1	Iterative development and validation process	11
4.1	System architecture and separation of analytical and application concerns	13
4.2	Context-level data flow diagram (DFD Level 0)	14
4.3	Major processes, external actors, and shared analytical stores (DFD Level 1)	15
4.4	Expanded simulation process (DFD Level 2 / operational flowchart)	16
4.5	Use-case diagram for planning, research, and technical integration	17
4.6	Sequence diagram for a live dashboard or API simulation	18
5.1	Observed temperature distribution and land-cover sample composition	20
5.2	Correlation structure of continuous variables	21
5.3	Spatial block cross-validation design	22
5.4	Dependency-oriented view of the thirteen numbered notebooks	24
6.1	Local dashboard capture: project overview and headline indicators	25
6.2	Observed LST in geographic coordinates, reconstructed from the V3 sample	26
6.3	Local dashboard capture: Heat Drivers and global SHAP importance	27
6.4	Local dashboard capture: intervention controls and simulation pipeline	28
6.5	Local capture of the default live simulation: paired before/after results	29
7.1	Logical entities and lineage across the file-based data model	30
8.1	Spatial distribution of the population-aware Heat Vulnerability Index	33
8.2	Saved notebook SHAP beeswarm: feature values and directional contributions	34
8.3	Top ten zones by mean of the best location-level cooling estimate	35
9.1	Reproduced spatial out-of-fold predictions and residuals	38

LIST OF TABLES

Number	Table title	Page No.
1.1	Scope and boundaries of the implemented project	3
1.2	Existing practice and the proposed response	4
2.1	Core libraries and their implemented roles	6
2.2	Documented data sources and analytical purpose	7
2.3	Execution requirements and recommendations	8
3.1	Functional requirements and implementation evidence	9
3.2	Quality requirements and current design response	10
3.3	Principal project risks and mitigation priorities	12
4.1	Representative use-case specification	17
4.2	Reusable module responsibilities	18
5.1	Dataset evolution and final analytical path	19
5.2	Final V3 descriptive audit	20
5.3	Deployed eleven-feature schema	21
5.4	Saved estimator and executed V3 training configuration	23
5.5	Notebook-reported V3 model comparison (mean fold metrics)	23
5.6	Notebook groups and generated products	24
6.1	Historical V2 and final V3 XGBoost results	27
7.1	Core data fields and constraints	30
7.2	Implemented API endpoints	31
7.3	Deployment and integration considerations	32
8.1	Leading thermal and vulnerability zones from saved V3 outputs	33
8.2	Leading mean absolute SHAP values	34
8.3	Notebook V3 scenarios across all 9,893 sampled locations	35
9.1	Executed data and inference checks	36
9.2	Executed API and integration test cases	37
9.3	Saved notebook results and report reproduction	38
10.1	Objective-level discussion of the implemented outcome	39
11.1	Proposed enhancements and their expected value	42
A.1	Indicative complexity of major operations	43
B.1	Publication details to be completed when applicable	44
C.1	Details to be completed after the institutional check	45

CHAPTER 1

INTRODUCTION

1.1 Project Introduction

UrbanHeat AI is a geospatial artificial-intelligence and machine-learning project for identifying urban heat-stress hotspots and comparing cooling strategies in Chhatrapati Sambhajinagar, Maharashtra. The repository develops an end-to-end path from sampled environmental observations to an interactive decision-support dashboard. Its central output is not merely a temperature map: the system connects predicted land surface temperature, explanatory feature attribution, population exposure, and location-specific intervention scenarios.

The study uses vegetation, built-up intensity, elevation, population, and land-cover information to model LST. It then aggregates point-level results into spatial zones so that planners can inspect where heat is concentrated, where people may be exposed, and which cooling action has greater model-based potential. The implementation combines an offline notebook pipeline with a Streamlit application, a shared Python inference module, and a FastAPI service.

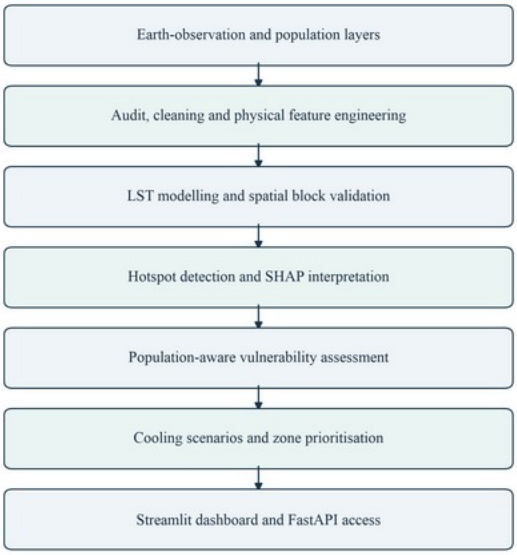


Figure 1.1: Decision-support workflow from environmental inputs to planning outputs

The project was developed in the context of Bharatiya Antariksh Hackathon 2026, Problem Statement PS-01, concerning the optimisation of urban heat mitigation and cooling strategies through AI/ML. This report treats the repository [16] as the primary evidence of implementation and distinguishes documented data provenance, executed notebook results, and proposed future extensions.

1.2 Background and Motivation

Urbanisation changes the surface energy balance. Vegetated and moist surfaces can dissipate energy through shading and evapotranspiration, while dense built-up materials may absorb and retain heat. The resulting thermal pattern varies with land cover, terrain, seasonal conditions, and the spatial arrangement of development. Remote sensing makes it possible to examine these differences over a wider area than isolated field measurements alone [1, 2].

Land surface temperature is a radiometric measure of the surface skin. It is related to, but not interchangeable with, near-surface air temperature or a complete human heat-stress index. LST can identify thermal contrasts and potential hotspots, whereas health-oriented heat risk also depends on humidity, wind, radiation exposure, activity, age, housing, and access to cooling. This distinction determines how the project's results should be interpreted.

1.3 Problem Statement

A city planning team needs a transparent method to locate persistent or intense surface-heat concentrations, understand their environmental correlates, account for population exposure, and compare possible cooling actions. A static map does not answer all of these questions. Nor does a high predictive score establish that changing a feature will cause a measured reduction in temperature.

The addressed problem is therefore to design and implement an integrated, reproducible decision-support framework that models LST from physical and land-cover indicators; validates predictions across spatially separated locations; explains model behaviour; identifies hotspot and vulnerability priorities; and exposes clearly bounded intervention simulations through a dashboard and an API.

1.4 Study Area

Chhatrapati Sambhajinagar, formerly Aurangabad, is the project's study area. The final sampled extent spans approximately 19.723–20.043° N and 75.142–75.587° E. These values describe the dataset's sampling envelope, not an authoritative municipal boundary. The data include built-up locations as well as cropland, grassland, trees, shrubs, bare surfaces, and water. Consequently, the analysis represents a sampled urban-region mosaic rather than every parcel within an administrative boundary.

1.5 Project Objectives

1. Prepare and audit a consistent geospatial dataset containing LST, environmental indicators, population, land cover, and coordinates.
2. Engineer an interpretable feature schema while excluding latitude and longitude from direct prediction.
3. Compare Linear Regression, Random Forest, and XGBoost using spatially grouped validation.
4. Detect high-temperature locations and aggregate their characteristics into planning zones.
5. Use SHAP to describe the model's feature dependence and communicate the limits of attribution.
6. Combine heat hazard and population exposure in a transparent Heat Vulnerability Index.
7. Compare greening and cool/permeable-surface scenarios, retaining both positive and negative local effects.
8. Deliver reusable inference, interactive visualisation, programmatic API access, and downloadable analytical outputs.

1.6 Project Scope

Table 1.1: Scope and boundaries of the implemented project

Included in the current project	Outside the current implementation
Sampled LST prediction and observed-LST hotspot analysis	Continuous live satellite ingestion or city-wide real-time monitoring
Spatial validation, model comparison, and SHAP interpretation	Validated causal estimation of intervention effectiveness
Population-proxy vulnerability and zone ranking	Health diagnosis or complete socioeconomic vulnerability assessment
Notebook scenarios and five-control interactive simulator	Engineering design, cost optimisation, or automatic intervention approval
Streamlit views, Folium maps, FastAPI endpoints, CSV products	Authentication, multi-tenant administration, or production service-level guarantees

The scope is deliberately limited to decision support. The system helps organise evidence and compare alternatives; municipal departments and domain experts remain responsible for field verification and implementation decisions.

1.7 Existing System and Identified Gaps

Conventional heat assessment commonly begins with a thermal raster, field survey, or manually prepared map. These approaches are useful for describing temperature patterns, but their outputs may remain separated from model validation, population data, and intervention comparison. A planner may consequently need to move between GIS layers, spreadsheets, and independent analytical scripts before forming a priority list.

Table 1.2: Existing practice and the proposed response

Existing limitation	Response implemented in UrbanHeat AI
Heat location is shown without a common predictive workflow	An ordered physical-feature schema and saved XGBoost estimator
Nearby random test samples can overstate generalisation	Whole geographic blocks held out through GroupKFold
Feature importance is difficult to interpret	Global SHAP values and model-driver views
Thermal priority is confused with human exposure	Separate hotspot and population-aware vulnerability rankings
Cooling actions are discussed without a common comparison basis	Reproducible notebook scenarios and a shared live simulator
Analytical outputs remain disconnected from users	Dashboard tables, maps, charts, API responses, and CSV exports

1.8 Proposed System and Report Organisation

The proposed system links data preparation, model development, explainability, and planning outputs through versioned files. Chapters 2 and 3 describe technologies and requirements. Chapter 4 presents the architecture and design diagrams. Chapters 5 and 6 explain the analytical and application implementation. Chapter 7 documents data and API contracts. Chapters 8 and 9 present analytics and testing. Chapters 10 and 11 discuss the results, limitations, conclusion, and future scope. The appendices cover feasibility, complexity, publication status, and the institutional plagiarism-check placeholder.

CHAPTER 2

TECHNOLOGY STACK

2.1 Technology-Stack Overview

The implementation uses Python as the common language for data processing, model development, inference, visualisation, and service delivery. The stack is file-based rather than database-centred: CSV files carry records between processing stages, a Joblib pickle stores the fitted estimator, and HTML files provide standalone interactive maps. This arrangement suits a research prototype with a moderate dataset and a reproducible notebook workflow.

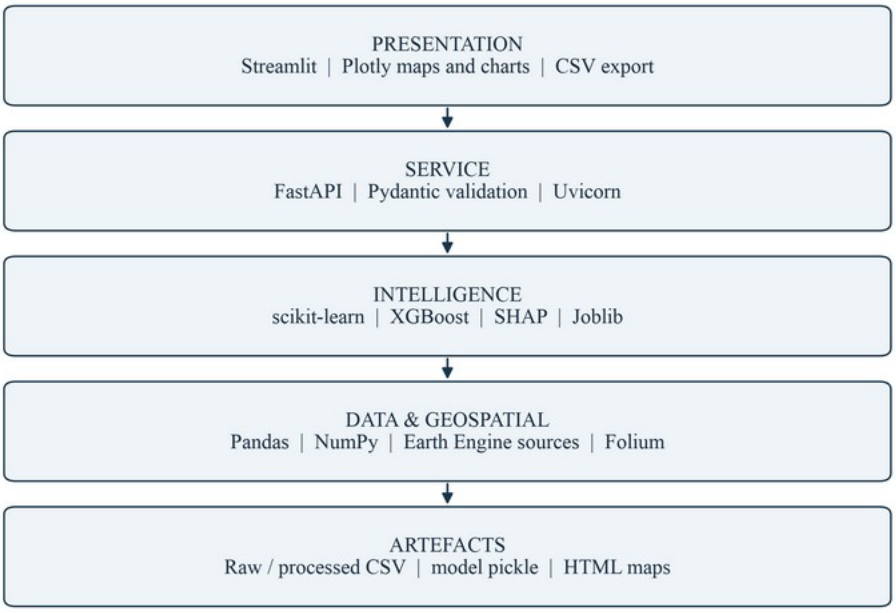


Figure 2.1: Layered technology stack used by the project

The dashboard and API are separate user-facing surfaces, but live prediction and intervention calculations converge on src/inference.py. This reduces the risk of different numerical logic appearing in different interfaces. Precomputed dashboard views are loaded from outputs/dashboard_data, while the interactive simulator loads the featured dataset and trained model.

2.2 Python and Data-Processing Libraries

Python provides a consistent environment for geospatial tabular analysis and application development. Pandas supplies DataFrame operations, CSV input/output, grouping, joins, descriptive statistics, and categorical encoding. NumPy supplies vectorised numerical operations, clipping, normalisation, array calculations, and metric computation. These libraries allow the notebook pipeline and the deployed inference code to use the same representation of a sampled location.

Table 2.1: Core libraries and their implemented roles

Technology	Role in the repository
Python	Common language for notebooks, reusable modules, dashboard, and API
Pandas	Data audit, cleaning, feature tables, zone aggregation, CSV interchange
NumPy	Vector operations, clipping, spatial block calculations, numerical metrics
scikit-learn	Linear Regression, Random Forest, GroupKFold, regression metrics
XGBoost	Final nonlinear LST regression model and scenario re-prediction
SHAP	Tree-model feature attribution on a reproducible 1,000-row sample
Joblib	Persistence and loading of the fitted XGBoost estimator
Matplotlib	Notebook charts, analytical plots, and saved SHAP visualisation

2.3 Model-Development Environment

Jupyter notebooks divide the work into auditable stages. They retain code, explanatory markdown, and saved outputs, making it possible to trace a reported figure to its generating operation. The repository contains thirteen numbered notebooks. Notebook 05 includes the final land-cover-aware V3 model training and comparison; notebooks 09 and 10 preserve earlier four-feature experiments. The distinction is important when reproducing the deployed model.

2.4 Earth-Observation and Geospatial Technologies

The repository documentation identifies Google Earth Engine as the source-data integration environment and QGIS as a supporting geospatial tool. The checked-in analytical pipeline consumes exported CSV files; it does not include the original Earth Engine acquisition script or a QGIS project. Accordingly, this report describes those technologies as documented source or supporting tools, not as steps independently rerun during report preparation.

Table 2.2: Documented data sources and analytical purpose

Parameter / source	Documented resolution	Use
LST: Landsat 8/9 Collection 2 Level 2	30 m thermal product	Regression target and observed heat mapping
NDVI: Sentinel-2 SR Harmonized	10 m	Vegetation condition proxy
NDBI: Sentinel-2 SR Harmonized	10–20 m bands	Built-up / impervious-surface proxy
Elevation: SRTM	30 m	Terrain context
Population: WorldPop India 2020	100 m	Population-exposure proxy
Land cover: ESA WorldCover classes in V3 notebooks	Source class layer; sampling exported to CSV	Categorical surface context

Native raster resolution does not imply that every exported sample independently measures conditions at that resolution. Resampling, acquisition timing, clouds, and aggregation can affect the information content. The CSV does not preserve scene identifiers or acquisition dates, so those details cannot be verified from the repository alone.

2.5 Folium, Branca, and Plotly

Folium and Branca create the standalone Leaflet-based heat and vulnerability maps in the notebook outputs [15]. Plotly powers the current Streamlit charts [15] and sampled point maps. Streamlit-folium is listed in requirements as an integration dependency, although the current app.py primarily renders maps with Plotly. External map tiles and web fonts require network availability.

2.6 Application, API, and Deployment Technologies

Streamlit implements the interactive dashboard in app.py [14]. It provides navigation, filters, sliders, metric cards, tables, charts, caching, and CSV downloads. FastAPI exposes the shared inference operations in api.py, with Pydantic request schemas and automatically generated OpenAPI documentation. Uvicorn is the configured ASGI server. The repository README also identifies a deployed Streamlit application; the screenshots in this report were captured from a local run of the checked-in code.

2.7 Software and Hardware Requirements

Table 2.3: Execution requirements and recommendations

Item	Requirement / recommendation
Operating system	Windows, Linux, or macOS with a supported Python environment
Python	Python 3.10 or newer; report checks used Python 3.11
Memory	8 GB minimum for the current dataset; 16 GB preferred for retraining and SHAP
Processor / storage	Modern multi-core CPU; approximately 1 GB free for environment and working outputs
Development tools	Jupyter-compatible notebook environment, Git, and a code editor
Application packages	Dependencies in requirements.txt, with compatible versions pinned for deployment
Browser / network	Modern browser; internet access for external map tiles and hosted deployment
Optional supporting tools	Google Earth Engine access for source acquisition; QGIS for GIS inspection

No GPU is required for the current tabular workflow. The requirements file does not pin versions. During validation, Plotly 7 no longer exposed the scatter_mapbox function used by app.py; the dashboard was therefore exercised with Plotly 5.24.1. A deployment lockfile and compatibility tests are recommended rather than assuming that an unpinned installation will remain reproducible.

CHAPTER 3

SYSTEM ANALYSIS AND REQUIREMENTS

3.1 Stakeholders and Functional Requirements

The principal users are planners, researchers, and technical integrators. Planners need interpretable maps and priority lists; researchers need data definitions, validation methods, and reproducible exports; integrators need stable request and response schemas. The current system does not implement separate authenticated administrator and citizen accounts. Its functional requirements therefore describe analytical and public decision-support capabilities rather than user-management modules.

Table 3.1: Functional requirements and implementation evidence

ID	Required capability	Implementation evidence
FR-01	Load and audit the source dataset	Notebooks 01–02; audit and cleaning CSV summaries
FR-02	Prepare ordered model features	Notebook 04; model-ready V3 CSV; feature-order file
FR-03	Train and compare regression models	Notebook 05; model comparison CSV
FR-04	Validate on held-out spatial groups	GroupKFold over 334 V3 zones
FR-05	Display and rank heat hotspots	Notebooks 06 and 11; hotspot dashboard page
FR-06	Explain model feature contributions	Notebook 07; Heat Drivers page
FR-07	Estimate population-aware vulnerability	Notebook 12; vulnerability page and zone CSV
FR-08	Compare and customise cooling scenarios	Notebook 08; shared inference; Cooling Strategy page
FR-09	Expose predictions and simulation through HTTP	FastAPI /predict and /simulate endpoints
FR-10	Export results for external analysis	Dashboard download button and CSV output files

3.2 Non-Functional Requirements

Table 3.2: Quality requirements and current design response

Quality	Requirement and design response
Reproducibility	Preserve source versions, processing notebooks, feature order, model parameters, and validation outputs.
Consistency	Use a single inference module for dashboard and API predictions and scenario calculations.
Usability	Provide readable labels, filters, priority tables, progressive disclosure of assumptions, and a clear navigation structure.
Performance	Cache prepared CSV views and loaded model resources; downsample interactive maps to 2,000 points while retaining full data in exports.
Reliability	Check required files and features; return structured API errors for empty requests and invalid control ranges.
Transparency	Expose model metrics, feature attribution, threshold definitions, and scenario assumptions.
Maintainability	Separate raw data, processed data, models, reports, reusable modules, and user interfaces.
Portability	Use Python packages and file-based artefacts that can run locally or on a compatible hosted service.
Security boundary	Validate request schemas, but add authentication, rate limiting, transport policy, and upload restrictions before sensitive production use.

These are design requirements, not a claim of production certification. Concurrent-user load, denial-of-service resistance, formal accessibility conformance, and long-term hosted availability were not established by the repository checks. The report separates observed behaviour from qualities that require additional testing.

3.3 Feasibility Analysis

Technical feasibility is supported by the existing execution of the notebook pipeline, the saved estimator, and the working shared inference path. The dataset is small enough for CPU-based tabular modelling. Economic feasibility follows from the use of open-source libraries and CSV storage, although hosting, source-data access, field validation, and maintenance can still incur cost. Operational feasibility depends on whether decision-makers understand the difference between LST, air temperature, exposure, and model-based cooling potential.

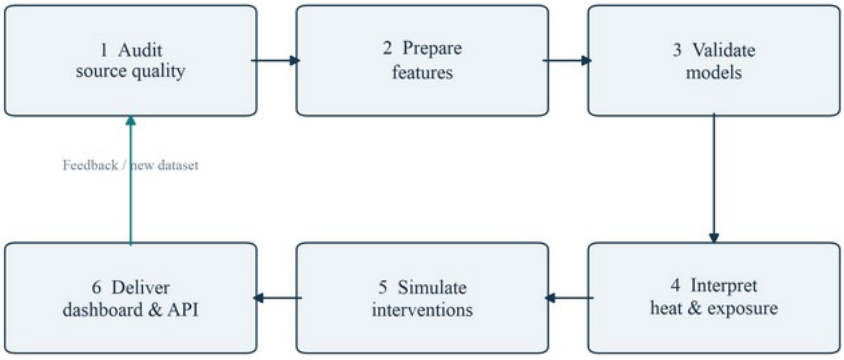


Figure 3.1: Iterative development and validation process

3.4 Development Process and Schedule Logic

An incremental process is appropriate because each analytical stage produces an artefact that can be checked before the next stage begins. Dataset V1 supports initial exploration, V2 extends the sampled dataset and feature experiments, and V3 adds categorical land-cover context. The dashboard can initially consume prepared CSV files; live simulation and API access can then be added without changing the underlying analytical definitions. Feedback from validation and interpretation informs the next iteration.

The repository does not contain an approved week-by-week project calendar. A defensible schedule should therefore be derived from the actual task dependencies—data audit, feature preparation, validation, analysis, application integration, testing, and documentation—rather than presenting invented completion dates.

3.5 Constraints, Assumptions, and Risks

Table 3.3: Principal project risks and mitigation priorities

Risk	Potential effect	Mitigation / current boundary
Unverified acquisition metadata	Seasonal or resolution mismatch may be hidden	Recover Earth Engine scripts, dates, masks, and sampling design
Spatial dependence	Random-split scores may be optimistic	Hold out complete geographic groups; consider buffered validation
Correlated predictors	Attribution may be shared or unstable	Inspect NDVI–NDBI relationship and test alternative feature sets
Population-proxy limitations	Exposure ranking may omit social sensitivity	Add demographic, housing, health, and cooling-access indicators
Uncalibrated intervention coefficients	Scenario outputs may be mistaken for causal forecasts	Label assumptions and calibrate against field observations
Small spatial zones	Unstable means and extreme ranks	Display sample counts and require minimum-support rules
Unpinned dependencies	Application may fail after package upgrades	Pin compatible versions and run interface smoke tests
Open service endpoints	Resource abuse or unintended public access	Add authentication, quotas, monitoring, and deployment controls

3.6 Acceptance Criteria

The analytical workflow is considered internally consistent when the model receives the saved feature order, spatial groups do not cross training and test folds, reported metrics are traceable, and dashboard/API simulations use identical logic. Planning outputs must preserve coordinates and zone identifiers, expose uncertainty and assumptions, and remain downloadable. These criteria are evaluated in Chapter 9; they do not replace external scientific or municipal validation.

CHAPTER 4

SYSTEM DESIGN AND ARCHITECTURE

4.1 Overall System Architecture

The architecture separates offline analytical preparation from online presentation and inference. Offline notebooks transform the source CSV into model-ready data, validation reports, hotspot records, scenario outputs, vulnerability scores, and dashboard-ready tables. The dashboard reads these prepared outputs for fast visual exploration. For a new live scenario, it loads the saved model and featured V3 dataset and calls the canonical inference module. The API follows the same path for programmatic requests.

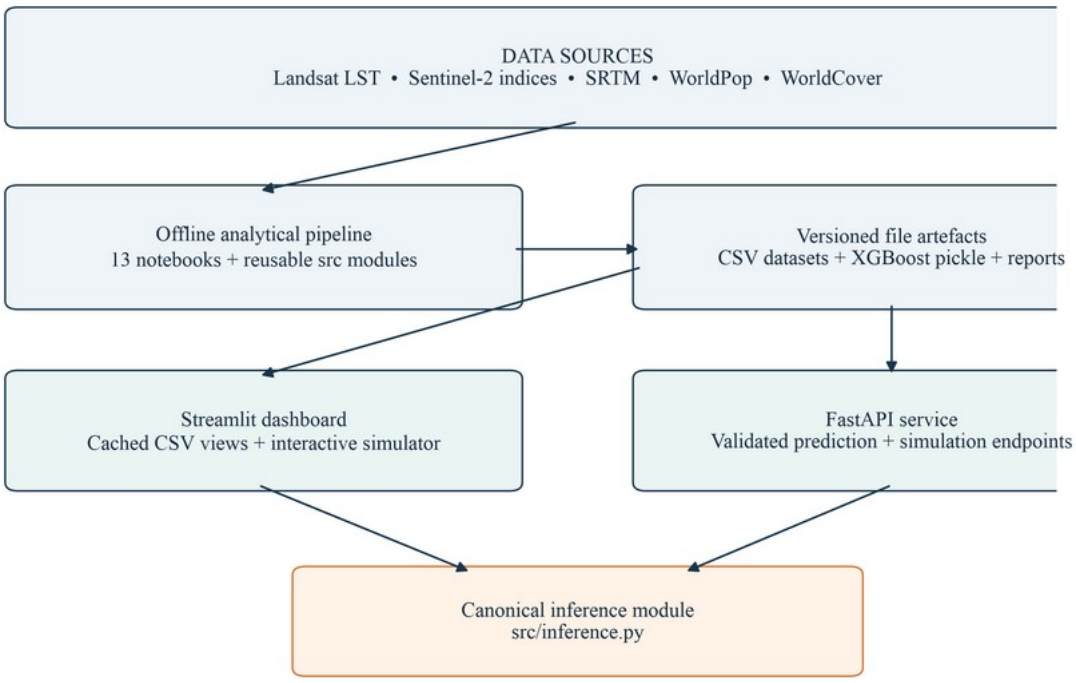


Figure 4.1: System architecture and separation of analytical and application concerns

This design has two important consequences. First, prepared visualisations remain available without retraining the model on every page visit. Second, live scenario calculations are not reimplemented separately in the API and dashboard. The architecture is a modular monolith with file-based persistence, not a microservice system or a transactional database application.

4.2 Context-Level Data Flow Diagram

The Level 0 diagram treats UrbanHeat AI as one decision-support process. Users provide filters or intervention settings and receive maps, model summaries, priority tables, or exported records. External technical clients exchange feature records and scenario parameters for JSON responses. The system depends on prepared CSV files and a fitted estimator; it does not acquire new satellite imagery during an ordinary dashboard request.

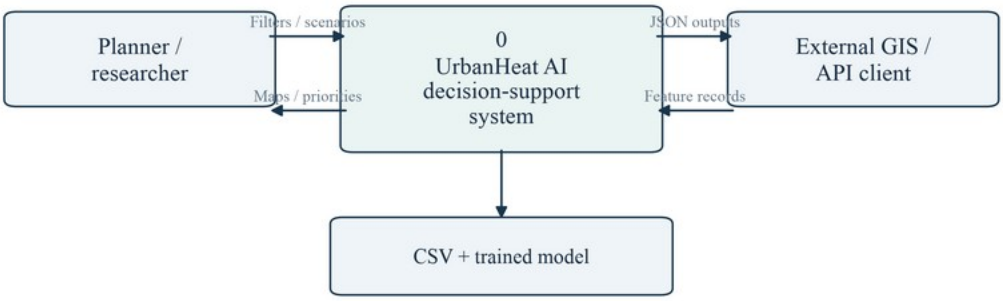


Figure 4.2: Context-level data flow diagram (DFD Level 0)

4.3 Level 1 Data Flow Diagram

At the next level, data preparation, model validation, heat and vulnerability analysis, and scenario communication are separated. Intermediate files form the contracts between these processes. This makes the workflow inspectable: an analyst can open a processed dataset or report without running the user interface. The stored model is produced offline and reused by downstream analysis and application services.

4.4 Level 1 Processes and Data Stores

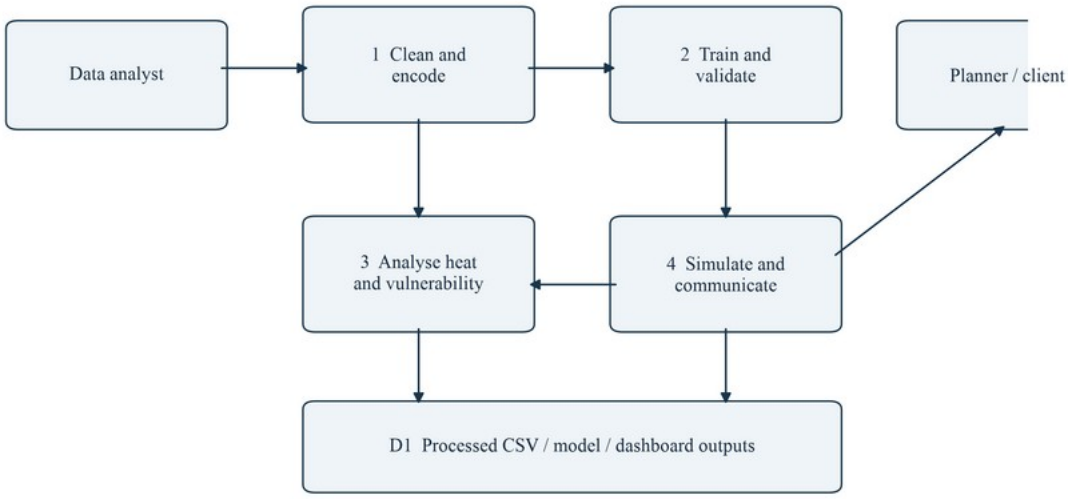


Figure 4.3: Major processes, external actors, and shared analytical stores (DFD Level 1)

Process 1 removes export metadata and creates categorical features. Process 2 evaluates candidate models and writes the final estimator and comparison results. Process 3 derives hotspot, exposure, and vulnerability products from a common set of locations. Process 4 compares interventions and exposes results through maps, tables, JSON, and CSV. The data-store symbol represents a collection of versioned files rather than a single relational database.

The principal join keys are the row-level location records and the derived Zone_ID. Coordinates support spatial grouping and visualisation. Zone_ID is created from latitude and longitude offsets using a 0.02-degree grid in the final notebook workflow. Because all downstream products retain this identifier, hot intensity, vulnerability, and cooling potential can be compared for the same zone.

4.5 Cooling-Simulation Flowchart

The simulation is a deterministic feature-modification and re-prediction process. It begins with validated control values, copies the baseline dataset, changes environmental indicators according to explicit coefficients, and constrains the resulting values. Land-cover fractions are normalised to sum to one. The same trained estimator then predicts baseline and modified LST, allowing a paired comparison at each sampled location.

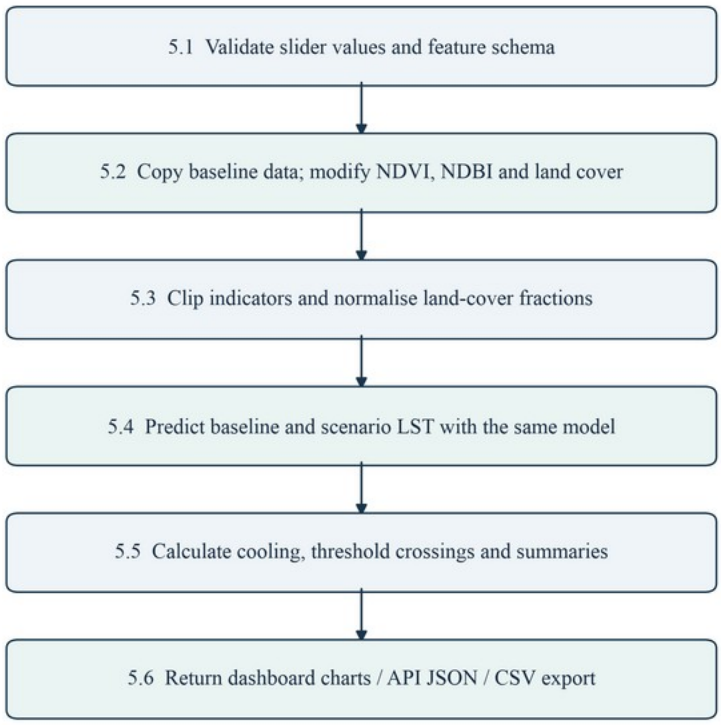


Figure 4.4: Expanded simulation process (DFD Level 2 / operational flowchart)

The flowchart also identifies where responsible interpretation is required. A numerical temperature difference is not a measured intervention effect. Its meaning depends on the assumed indicator changes, the model's learned relationships, and whether the modified feature combination remains physically plausible. The system therefore reports negative changes and exposes its assumptions rather than forcing every intervention to appear beneficial.

4.6 Use-Case Design

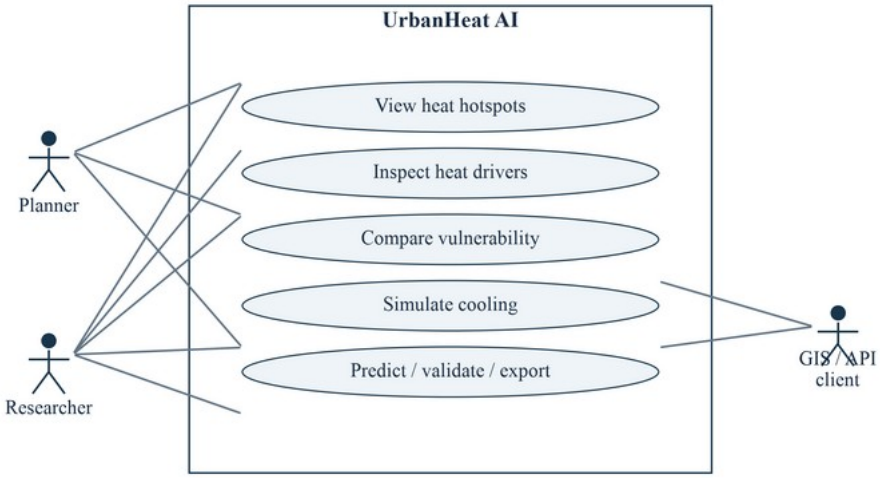


Figure 4.5: Use-case diagram for planning, research, and technical integration

A planner primarily inspects spatial priorities and compares possible interventions. A researcher additionally examines model metrics, feature attribution, data definitions, and exported records. A GIS or API client uses the HTTP interface to obtain predictions and simulation summaries. These are logical roles, not authenticated account types: the checked-in application does not enforce role-based access control.

Table 4.1: Representative use-case specification

Field	Description
Use case	Compare a cooling intervention mix
Primary actor	Planner or researcher
Precondition	Featured V3 data and the trained estimator are available
Main flow	Set five controls → run simulation → inspect before/after metrics → review local effects → export CSV
Alternative flow	Missing artefact or schema → display an error and stop the calculation
Postcondition	A scenario summary and modified location-level records are produced; the source data remain unchanged

4.7 Sequence and Module Design

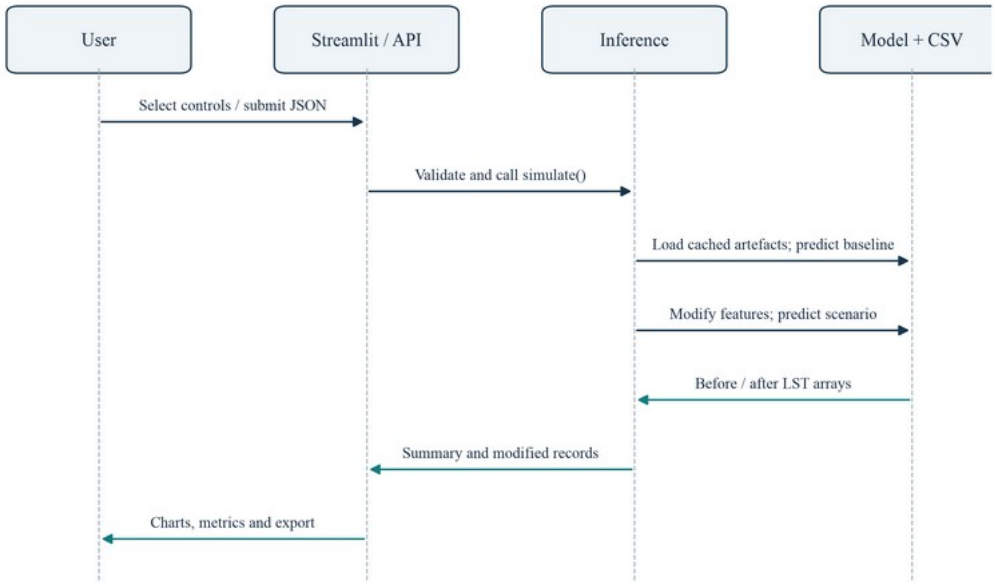


Figure 4.6: Sequence diagram for a live dashboard or API simulation

The sequence begins with user input or a JSON request. Streamlit controls or Pydantic schemas establish the input values. The interface calls the shared simulation function, which loads or receives cached resources, predicts the baseline, applies feature changes, predicts the scenario, and returns paired results. The interface then renders summaries and enables export. No source dataset or model is rewritten by this interaction.

Table 4.2: Reusable module responsibilities

Module	Responsibility
src/inference.py	Canonical feature order, model loading, classification, indicator changes, prediction, and simulation
src/model.py	Training reference and an alternative spatial-CV helper; not identical to the final notebook settings
src/cleaning.py	Earlier duplicate removal and numeric median-imputation helpers
src/features.py / src/eda.py	Exploratory composite features, summaries, correlations, and histograms
src/hotspot.py	Wrapper around the canonical five-tier classifier
app.py / api.py	Streamlit presentation and FastAPI HTTP interface

CHAPTER 5

DATA AND MACHINE-LEARNING IMPLEMENTATION

5.1 Dataset Versions and Source Provenance

The repository contains three generations of input data. V1 provides an initial 5,000-row sample. V2 expands the sample to 9,893 locations and supports physical-indicator and composite-feature experiments. V3 retains the same sample count while adding land-cover class codes. The final deployed estimator uses V3 with eleven inputs. Version names in older notebooks and model filenames are not always reliable indicators of their contents, so the report identifies the actual input file and feature schema.

Table 5.1: Dataset evolution and final analytical path

Version / file	Rows	Principal role
Sambhajinagar_UHI_Dataset_V1.csv	5,000	Initial exploration and early processed outputs
Sambhajinagar_Dataset_V2 (1).csv	9,893	Expanded physical-indicator experiments
Sambhajinagar_Dataset_V3_Final.csv	9,893	Final source with seven land-cover classes
model_ready_uhi_v3.csv	9,893	Eleven predictors, LST target, and spatial coordinates
featured_uhi_v3.csv	9,893	Readable land-cover labels plus encoded features for live simulation

The repository README attributes the environmental layers to Landsat, Sentinel-2, SRTM, and WorldPop [3–7]. The V3 notebooks interpret class codes 10, 20, 30, 40, 50, 60, and 80 as ESA WorldCover classes [8]. The CSV itself does not contain acquisition dates, cloud-mask details, sensor scene identifiers, or the original Earth Engine export code. These are provenance limitations rather than details that can be inferred safely.

5.2 Data Audit, Cleaning, and Exploratory Analysis

Notebook 01 audits the raw V3 table. It contains 9,893 rows and ten columns, with no missing cells or duplicate rows. Notebook 02 removes system:index and .geo, retaining eight columns. Land-cover codes are checked against the expected set. No median imputation is needed for the final V3 data, although the reusable cleaning module contains a median-imputation helper from the broader workflow.

Table 5.2: Final V3 descriptive audit

Variable	Mean	Minimum	Maximum
LST (°C)	43.4636	27.7622	51.9566
NDVI	0.2411	−0.1289	0.7517
NDBI	0.0299	−0.4334	0.4013
Elevation (m)	621.37	483	926
Population proxy	12.3785	0.6679	230.7747

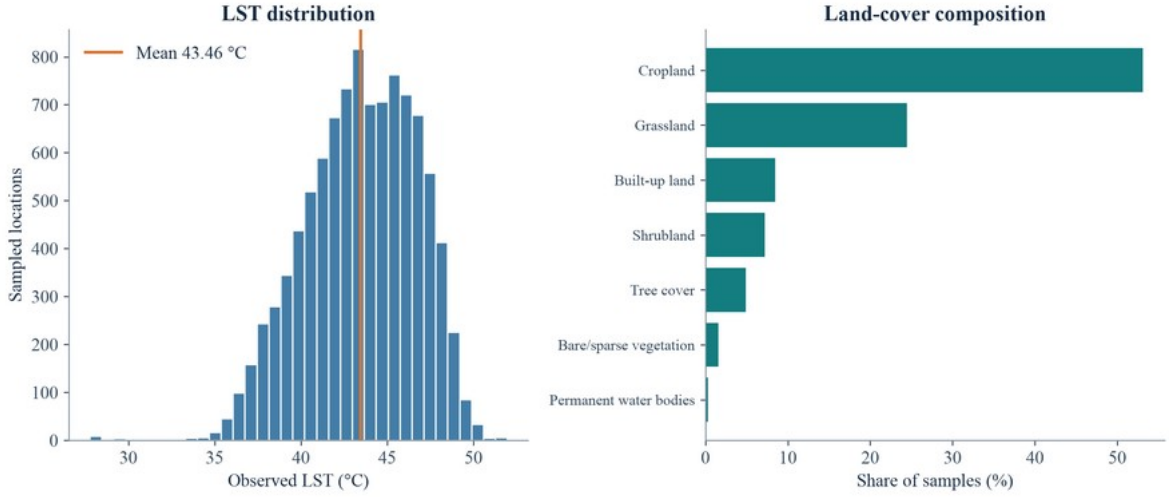


Figure 5.1: Observed temperature distribution and land-cover sample composition

Cropland accounts for 53.10% of samples and grassland for 24.47%; built-up land accounts for 8.48%. The class distribution is a property of the sample, not a verified estimate of municipal land-area shares. Water has only 29 observations, so class-specific conclusions for water are particularly uncertain.

5.3 Feature Engineering and Leakage Control

The final model uses four continuous variables and seven binary land-cover indicators. The categorical LandCover field is mapped to readable labels and one-hot encoded. Each row belongs to exactly one class, so its seven encoded values sum to one. The original class code is not used as an ordinal numeric predictor. Latitude and longitude remain in the model-ready file for spatial grouping and mapping, but are excluded from X.

Table 5.3: Deployed eleven-feature schema

Feature group	Fields and treatment
Vegetation / built-up	NDVI and NDBI, retained as continuous indices
Terrain / exposure	Elevation and Population, retained as continuous variables
Land-cover indicators	Bare/sparse vegetation; Built-up land; Cropland; Grassland; Permanent water bodies; Shrubland; Tree cover
Target	Observed LST in degrees Celsius
Spatial-only fields	Latitude and Longitude; never passed as prediction features

The common index definitions are NDVI = (NIR − Red)/(NIR + Red) and NDBI = (SWIR − NIR)/(SWIR + NIR). Their values are imported from the source-export workflow rather than recomputed from raw bands in the checked-in pipeline. Earlier composite features—Green_Built_Ratio, Population_Heat_Index, and Elevation_Cooling_Index—are retained in V2 exploration but excluded from the final model to limit redundant transformations and preserve interpretability.

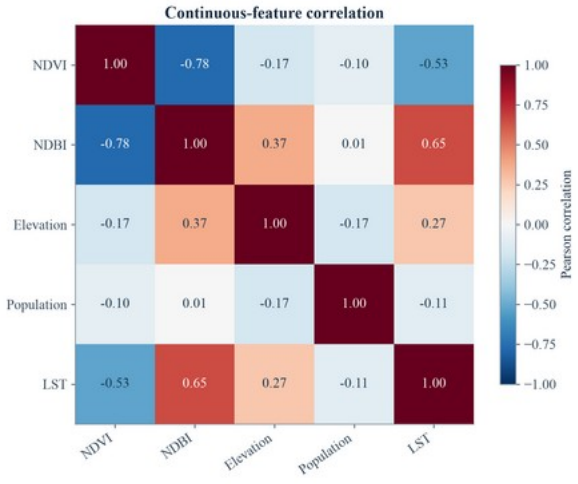


Figure 5.2: Correlation structure of continuous variables

5.4 Model Development and Spatial Validation

The scikit-learn implementations provide the linear and bagged tree baselines [13]. Linear Regression establishes an interpretable baseline. Random Forest tests a bagged ensemble of decision trees [9]. XGBoost tests boosted regression trees [10] capable of representing nonlinear relationships and interactions. The final V3 comparison appears in notebook 05, after the V3 feature-engineering section of notebook 04. All three use the same eleven-feature input schema and grouped validation design.

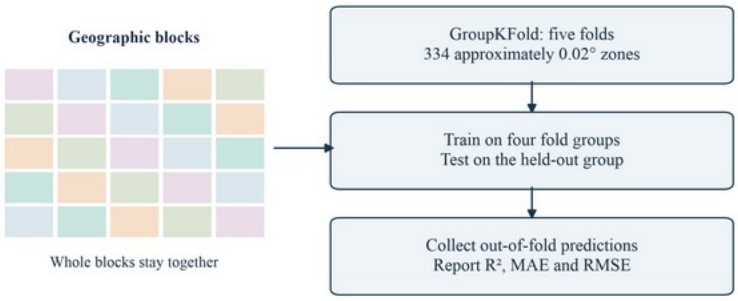


Figure 5.3: Spatial block cross-validation design

The executed V3 workflow forms 334 groups using floor((latitude − minimum latitude)/0.02) and the corresponding longitude expression. Five-fold GroupKFold assigns complete groups to folds, following the rationale for spatially structured validation [12]. Each location receives a prediction from a model that did not train on its spatial group. The notebook reports the arithmetic mean of the five fold-level R^2, MAE, and RMSE values. This is different from pooling all out-of-fold predictions before computing a metric.

R^2 measures explained variance relative to a mean baseline; MAE is the mean absolute temperature error; RMSE penalises larger errors more strongly. Grouped validation reduces neighbour leakage but does not eliminate adjacency effects at block edges or guarantee transfer to a different season or city. The final estimator is then fitted on all 9,893 rows for deployment; its in-sample predictions are not used as generalisation metrics.

5.5 Final XGBoost Configuration and Model Selection

Table 5.4: Saved estimator and executed V3 training configuration

Parameter	Value
Estimator / objective	XGBRegressor / reg:squarederror
Number of boosting rounds	500
Maximum tree depth	6
Learning rate	0.05
Minimum child weight	3
Row / column subsampling	0.8 / 0.8
Random seed	42
Input count / target	11 ordered features / LST (°C)

Table 5.5: Notebook-reported V3 model comparison (mean fold metrics)

Model	R^2	MAE (°C)	RMSE (°C)
Linear Regression	0.4896	1.8303	2.3039
Random Forest	0.5569	1.6858	2.1455
XGBoost	0.5691	1.6645	2.1163

XGBoost has the strongest reported values among the three candidates, although its margin over Random Forest is small. Selection is therefore based on the combined evidence of predictive performance, saved deployment integration, and compatibility with TreeExplainer rather than a claim of overwhelming superiority. The fitted object is saved as outputs/models/xgboost_v3_landcover_model.pkl, with feature order stored separately in a CSV.

The alternative helper in src/model.py uses a 10-by-10 quantile grid and different subsampling defaults. It is useful code, but it is not an exact reproduction of the final notebook configuration. This report uses the executed notebook and saved estimator as authoritative and identifies the difference as a maintainability issue.

5.6 Notebook Workflow and Reproducibility

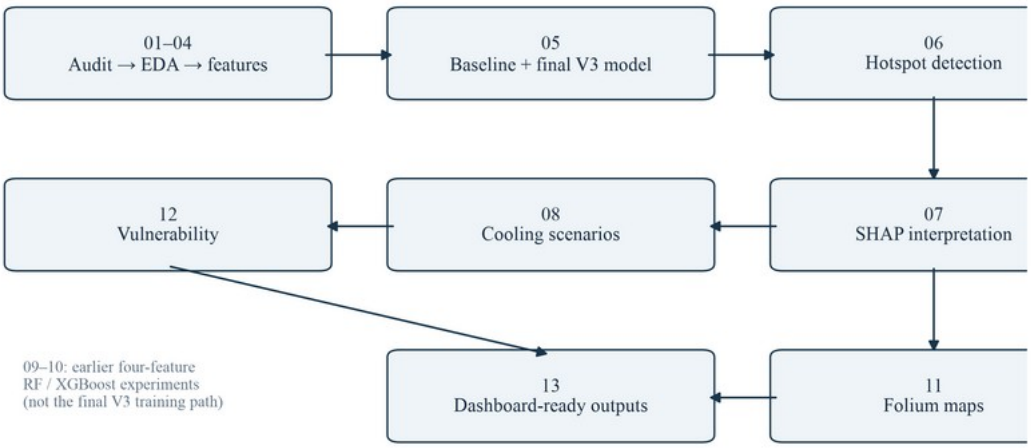


Figure 5.4: Dependency-oriented view of the thirteen numbered notebooks

Table 5.6: Notebook groups and generated products

Notebooks	Primary products
01–04	Audit, cleaning, exploratory analysis, composite-feature experiments, and final V3 encoding
05	V3 Linear Regression, Random Forest, XGBoost comparison; final estimator and feature order
06–08	Hotspot records and zone priorities; SHAP outputs; two notebook cooling scenarios
09–10	Earlier four-feature Random Forest and XGBoost experiments using V2 inputs
11–13	Folium heat/vulnerability maps, HVI products, and dashboard-ready CSV files

Notebooks should be run from the notebooks directory because their paths begin with ../data and ../outputs. The final V3 path is 01 → 02 → 03 → 04 → 05 → 06, followed by 07, 08, 11, 12, and 13 as required by their inputs. Notebook 12 consumes scenario output, and notebook 13 consumes the resulting analytical tables. A clean-environment run, dependency lockfile, and explicit acquisition script would strengthen reproducibility.

CHAPTER 6

DASHBOARD AND DECISION-SUPPORT IMPLEMENTATION

6.1 Dashboard Structure and Navigation

The Streamlit application presents the project as UrbanHeat AI: Thermal Intelligence Platform. A wide layout, persistent sidebar, dark visual theme, metric cards, and Plotly visualisations organise the analytical products. Navigation separates Project Overview, Urban Heat Hotspots, Heat Drivers, Heat Vulnerability, Cooling Strategy, Model Validation, and About Project. This structure allows a user to move from locating heat to understanding exposure and comparing action without opening the notebooks.

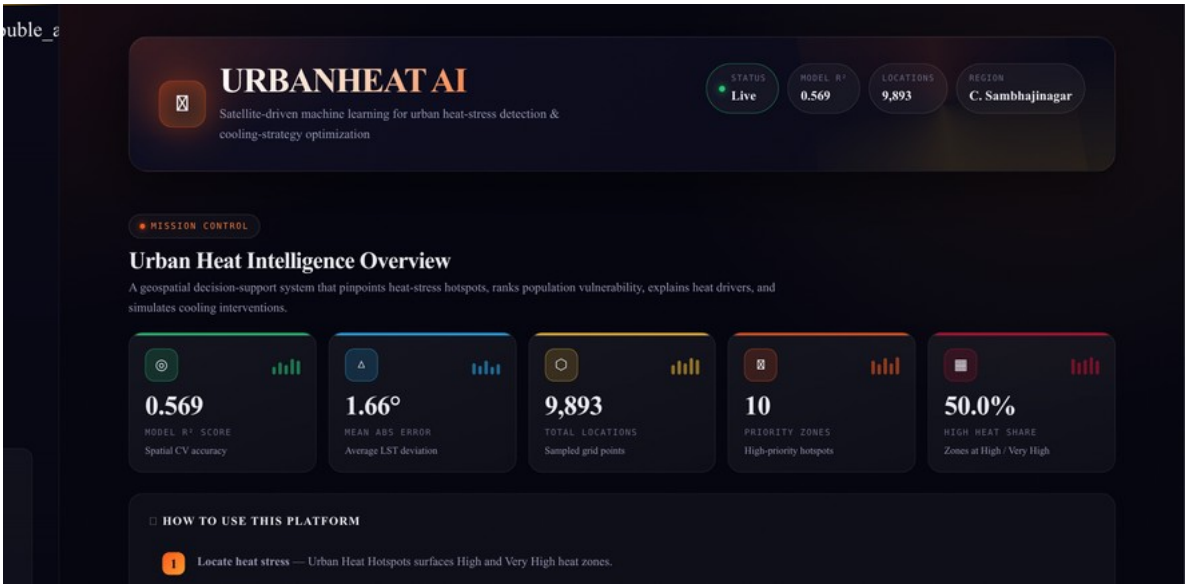


Figure 6.1: Local dashboard capture: project overview and headline indicators

The overview reads prepared KPI values, including the saved V3 model score, mean absolute error, number of sampled locations, and ten priority zones. It also provides a short decision workflow. The displayed 50.0% high-heat share refers to the upper two observed-LST quartiles, not the proportion of municipal land or population experiencing dangerous heat. Precise wording is necessary because the interface sometimes uses “zones” for point-level counts.

Dashboard data loading is cached. If a required file is absent, the application displays an instruction to run notebook 13 and stops. Map views use a reproducible sample of at most 2,000 locations for responsiveness; tables and exports retain the underlying full datasets.

6.2 Hotspot and Vulnerability Views

The hotspot view filters the prepared Heat_Category field and displays sampled locations on an interactive map. It also presents the top ten hotspot-priority zones. The saved V3 category is based on observed LST quartiles: Low, Moderate, High, and Very High. The upper category contains 2,472 points, approximately 24.99% of the sample. It is a relative dataset category rather than a medically validated heat-warning threshold.

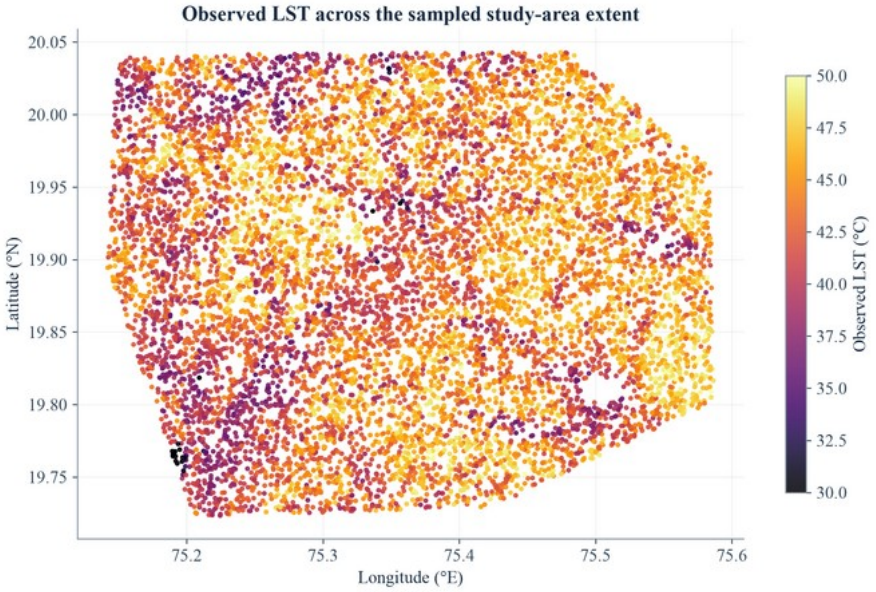


Figure 6.2: Observed LST in geographic coordinates, reconstructed from the V3 sample

The vulnerability view instead uses model-predicted baseline heat and population. Both are min –max normalised and combined with equal weights. Its four categories are HVI quartiles. The map helps identify places where heat and exposure overlap, while the priority table lists zone-level values. The Folium notebook maps provide standalone HTML alternatives with point popups and priority markers.

$$\mathbf{HVI = 0.5 \times norm(Predicted\ LST) + 0.5 \times norm(Population)}$$

$\text{norm}(x) = (x - \min(x)) / (\max(x) - \min(x))$; a constant field is mapped to zero. Zone HVI is the mean of its location-level HVI values.

The map screenshots requiring external basemap tiles were not used as evidence because the report-preparation environment could not retrieve those tiles. The geographic plot above is generated directly from repository coordinates and temperatures, without a basemap, and does not imply an administrative boundary.

6.3 Explainability and Model-Validation Views

The Heat Drivers page reads the saved SHAP importance table and presents a horizontal feature-attribution chart. It separates stronger overall contributors from weaker ones and explicitly notes that attribution is not causation. The Model Validation page reads the V2–V3 comparison CSV and lets the user select a numerical metric for charting. It explains that coordinates are excluded from predictors and used for spatial grouping.



Figure 6.3: Local dashboard capture: Heat Drivers and global SHAP importance

These views read precomputed SHAP and comparison files, so they remain responsive and consistent with the saved analysis. When the model or dataset changes, the files must be regenerated together; a visible model identifier and generation timestamp would make this dependency clearer.

Table 6.1: Historical V2 and final V3 XGBoost results

Dataset / design	R^2	MAE (°C)	RMSE (°C)
V2: four physical features; 10×10 quantile-grid validation	0.510	1.777	2.259
V3: eleven features; 334 fixed-width 0.02° groups	0.5691	1.6645	2.1163

V2 is historical context, not a controlled ablation: features and spatial grouping differ. V3 is deployed. The chart is a version history, not proof that land cover alone caused the improvement.

6.4 Interactive Cooling-Strategy Simulator

The Cooling Strategy page exposes five controls: green-cover increase, tree plantation, cool-roof adoption, permeable surface, and built-up reduction. Each accepts a value from 0 to 100. Defaults are 20, 25, 15, 15, and 10 respectively. Pressing Run AI Simulation loads cached resources, validates required columns, calls the shared inference function, and renders before/after metrics and location-level changes.

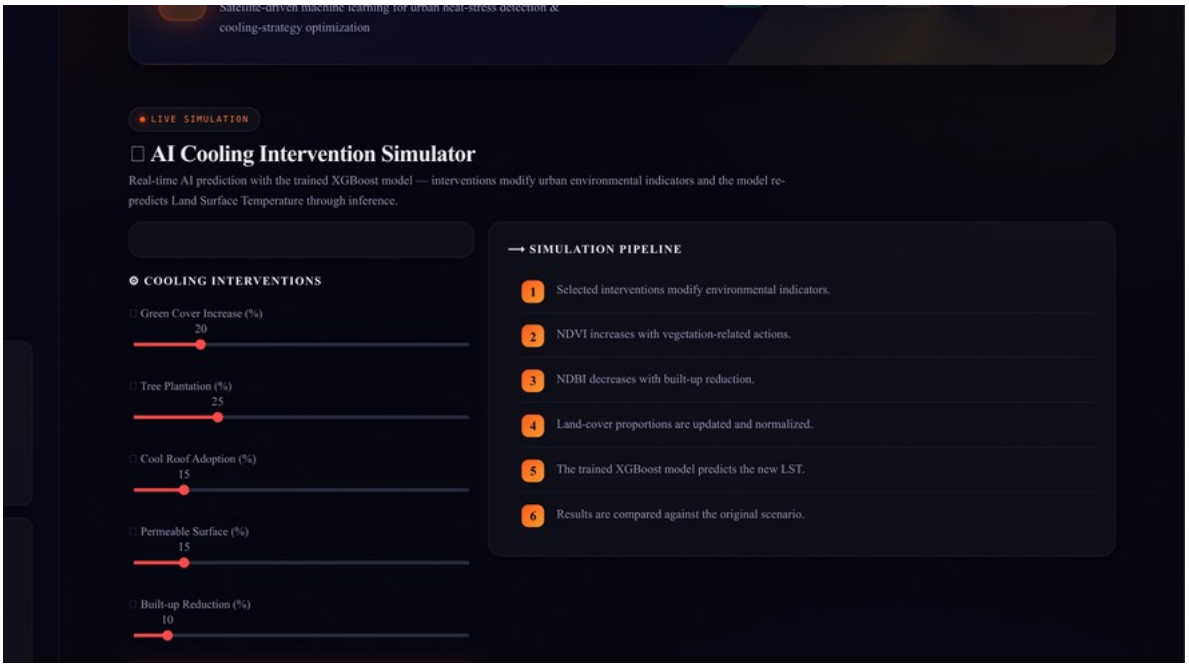


Figure 6.4: Local dashboard capture: intervention controls and simulation pipeline

An expandable methodology panel lists every sensitivity coefficient. Vegetation actions increase NDVI; built-up reduction decreases NDBI; selected actions alter land-cover fractions. Values are clipped and the land-cover columns are normalised. The simulator is an assumption-driven what-if engine, not an optimisation solver: it does not search a cost-constrained intervention portfolio or guarantee a globally best municipal plan.

Per percentage point, $\Delta NDVI = 0.0025g + 0.0035t + 0.0010p$ and $\Delta NDBI = -(0.0020b + 0.0010g + 0.0010t)$, where g, t, p and b are green-cover, tree, permeable-surface and built-up-reduction controls. Cool roofs reduce the built-up fraction by 0.0010 per point; the remaining land-cover coefficients are defined in src/inference.py.

Results include average and maximum LST before and after, mean reduction, threshold-crossing counts, the share of locations with positive cooling, category summaries, an advisory panel, and a CSV download. The advisory text is rule-based, not a separate generative-AI model.

6.5 Simulator Results and Export Behaviour



Figure 6.5: Local capture of the default live simulation: paired before/after results

For the default control mix, the reproduced mean predicted LST changes from 43.4635 °C to 41.7862 °C, a mean reduction of 1.6773 °C. Approximately 90.10% of locations have a positive reduction. The count above the canonical 42 °C threshold changes from 7,003 to 5,147 locations. These are point counts, despite the dashboard label “extreme zones.” They are not counts of the 334 aggregated planning zones.

The live result must not be merged with the notebook scenario averages of 0.257 °C and 0.076 °C. The notebook applies multiplicative index changes in two separate alternatives; the live simulator applies additive, coefficient-based changes from five controls simultaneously. The download contains the modified feature values and paired predictions, allowing external inspection. Source files and the fitted model remain unchanged.

CHAPTER 7

DATA AND API DESIGN

7.1 Logical Data Model and File Contracts

The project uses a file-based logical data model. A sampled location is described by coordinates, environmental predictors, an observed target, and a land-cover class. Model artefacts define how those inputs are transformed into predictions. Prediction and scenario records retain location identity, while zone summaries support planning aggregation. Dashboard files are projections of these analytical products rather than independent sources of truth.

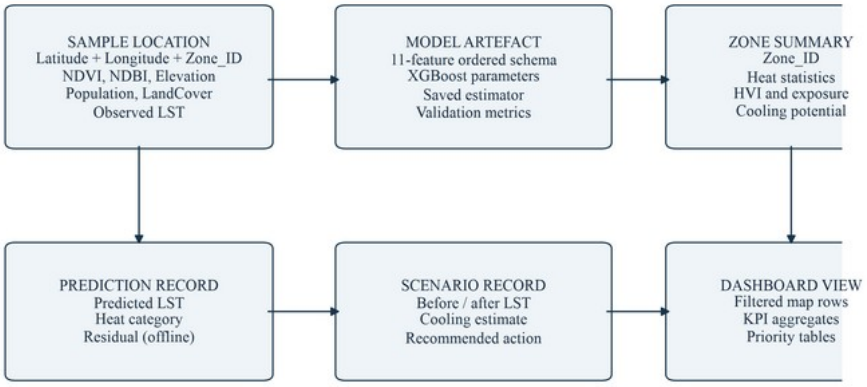


Figure 7.1: Logical entities and lineage across the file-based data model

Table 7.1: Core data fields and constraints

Field / group	Type and constraint	Purpose
Latitude, Longitude	Finite coordinates; retained outside model inputs	Mapping and spatial grouping
Zone_ID	Derived row_col string	Common aggregation identifier
NDVI, NDBI	Continuous indices; simulator clips to [−1, 1]	Vegetation and built-up context
Elevation, Population	Non-negative numeric values in the supplied data	Terrain and exposure context
Seven LandCover_* fields	Non-negative fractions summing to 1 after simulation	Categorical surface representation
LST / Predicted_LST	Degrees Celsius; observed and estimated values kept distinct	Target, prediction, and residual analysis

7.2 REST API Design

FastAPI exposes the fitted model through a small HTTP contract. Pydantic validates the eleven location features and the five intervention controls. The API imports src/inference.py rather than maintaining a second prediction implementation. Model and dataset loading use cached singleton functions so that repeated requests can reuse resources. Interactive OpenAPI documentation is available at /docs when the service is running.

Table 7.2: Implemented API endpoints

Method and path	Input	Response / behaviour
GET /health	None	Service status
GET /model-info	None	Model name, target, ordered features, validation description, metrics, and thresholds
POST /predict	locations: list of eleven-feature records	Count and list of predicted_lst_c and hotspot_category values
POST /simulate	Five percentage controls, each in [0, 100]	Control values, city-sample summary, and an assumptions note
GET /docs	None	Interactive OpenAPI interface generated by FastAPI

7.3 Request and Response Semantics

A prediction record uses the exact names NDVI, NDBI, Elevation, Population, and the seven LandCover_* fields. Pydantic aliases also accept LandCover_Built-up land and LandCover_Tree cover. An empty prediction list returns HTTP 400; missing required fields or out-of-range controls return HTTP 422. A missing simulation artefact is mapped to HTTP 503. Predictions are rounded to three decimals and simulation summaries to four decimals in the API response.

The API's /model-info response describes a 10-by-10 grouping design, while the authoritative V3 notebook comparison uses 334 fixed-width zones. Its numerical metric values match the saved V3 report, but the validation description should be aligned in a future code revision. This documentation discrepancy does not affect the prediction calculation itself.

7.4 Deployment, Security, and Integration Boundaries

Local execution requires the repository files, a compatible Python environment, and the dependencies in requirements.txt. Streamlit is launched with “python -m streamlit run app.py”. The API is launched with “uvicorn api:app --port 8000”. Paths are relative to the repository root, so both services should use that working directory. The README identifies Streamlit Community Cloud as the dashboard hosting route; no infrastructure-as-code or authenticated production API deployment is included.

Table 7.3: Deployment and integration considerations

Concern	Current state and recommended control
Transport	Use HTTPS at the hosting or reverse-proxy layer for non-local deployment
Authentication	No login or API-key enforcement is implemented; add access policy before restricted use
Resource protection	Add request-size limits, rate limiting, timeouts, and concurrency controls for public API service
Input trust	Load pickles only from a trusted repository artefact; do not accept user-uploaded model files
Data privacy	Current inputs are environmental grid samples; future health or household data require stronger governance
Dependency stability	Pin tested versions, especially Plotly, XGBoost, Pandas, and Streamlit
Observability	Record application errors, model version, request latency, and artefact hashes
Map availability	External tiles and fonts need network access; analytical CSV exports remain locally usable

The dashboard and API can be deployed as separate processes because they share files and code, not an in-memory session. A future multi-service deployment should distribute a versioned model bundle and immutable dataset together. Until then, a single trusted repository checkout is the simplest consistent deployment unit.

CHAPTER 8

DASHBOARD ANALYTICS AND RESULTS

8.1 Heat and Vulnerability Analytics

The final sample has a mean observed LST of 43.46 °C. The highest-priority thermal zones are ranked by mean observed LST and the share of upper-quartile hotspot points. The vulnerability ranking uses a different criterion: equal-weight predicted heat hazard and population exposure. The two top-ten lists have no overlapping Zone_IDs in the saved V3 results, demonstrating why thermal intensity and population-aware priority should not be treated as interchangeable.

Table 8.1: Leading thermal and vulnerability zones from saved V3 outputs

Rank	Hotspot zone	Mean LST (°C)	Vulnerable zone	Mean HVI
1	4_22	48.16	7_9	0.699
2	4_21	47.61	7_11	0.676
3	5_20	47.55	7_10	0.672
4	3_21	47.52	8_10	0.661
5	4_20	47.52	6_10	0.614

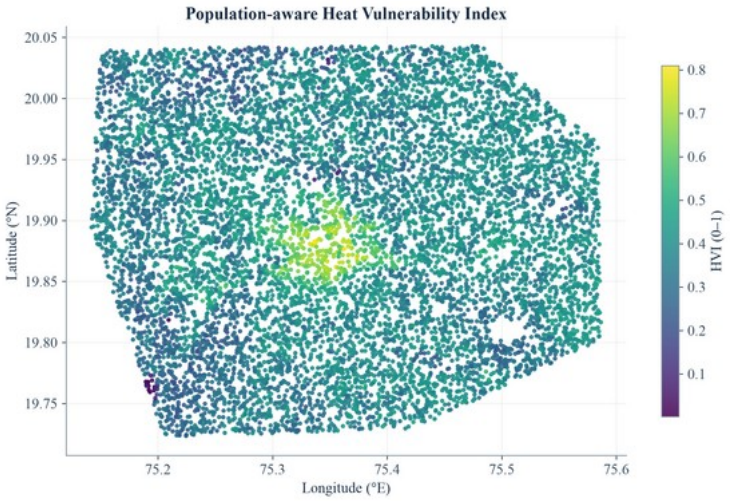


Figure 8.1: Spatial distribution of the population-aware Heat Vulnerability Index

Several thermal leaders contain few samples: zone 4_22 has six points and zone 0_15 has one. Their high mean values should trigger inspection, not automatic ranking confidence. Vulnerability leaders generally contain larger populations, but the population field remains an exposure proxy rather than a direct count of heat-related health outcomes.

8.2 Heat-Driver and Explainability Analytics

SHAP TreeExplainer [11] is applied to a fixed sample of 1,000 V3 rows with random_state 42. Mean absolute SHAP values summarise the magnitude of each feature's contribution to predicted LST. NDBI is the strongest global contributor in the saved output, followed by elevation, NDVI, and population. Land-cover indicators contribute additional class-specific information. The magnitude ranking does not mean that every increase in a feature has the same effect at every location.

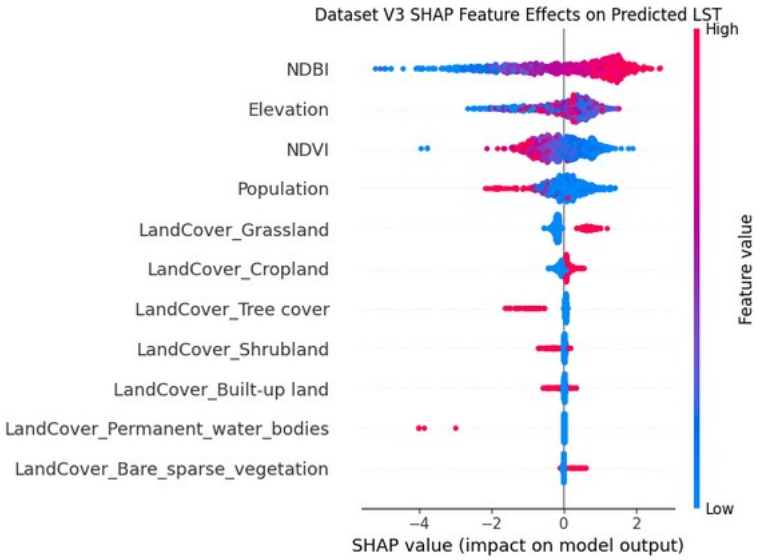


Figure 8.2: Saved notebook SHAP beeswarm: feature values and directional contributions

Table 8.2: Leading mean absolute SHAP values

Feature	Mean |SHAP|	Interpretation boundary
NDBI	1.2810	Strong model association; not a causal coefficient
Elevation	0.5802	Terrain and spatial context may be intertwined
NDVI	0.5350	Vegetation effects vary with surface and moisture conditions
Population	0.3770	Used as a predictor and separately as an exposure proxy
LandCover_Grassland	0.3203	Class-specific contribution relative to the sample

NDVI and NDBI have a Pearson correlation of approximately −0.775. Correlated predictors can share attribution, so the ranking should be read alongside feature effects, land-cover distributions, and validation results. SHAP explains the fitted model; it does not independently establish an environmental mechanism.

8.3 Cooling-Scenario and Priority Analytics

Table 8.3: Notebook V3 scenarios across all 9,893 sampled locations

Scenario	Indicator change	Mean cooling	Maximum cooling
Urban greening	NDVI × 1.20, capped at 1; NDBI × 0.90	0.257 °C	4.198 °C
Cool / permeable surfaces	NDBI × 0.80	0.076 °C	2.229 °C

The scenario code selects the larger predicted reduction at each location. Urban greening is selected for 6,272 locations and cool/permeable surfaces for 3,621. However, 7,646 locations have a positive best-scenario reduction, leaving 2,247 with zero or negative estimated benefit. A recommendation label is therefore not proof of cooling. The minimum greening effect is −5.217 °C and the minimum cool-surface effect is −3.006 °C, where negative cooling means predicted warming.

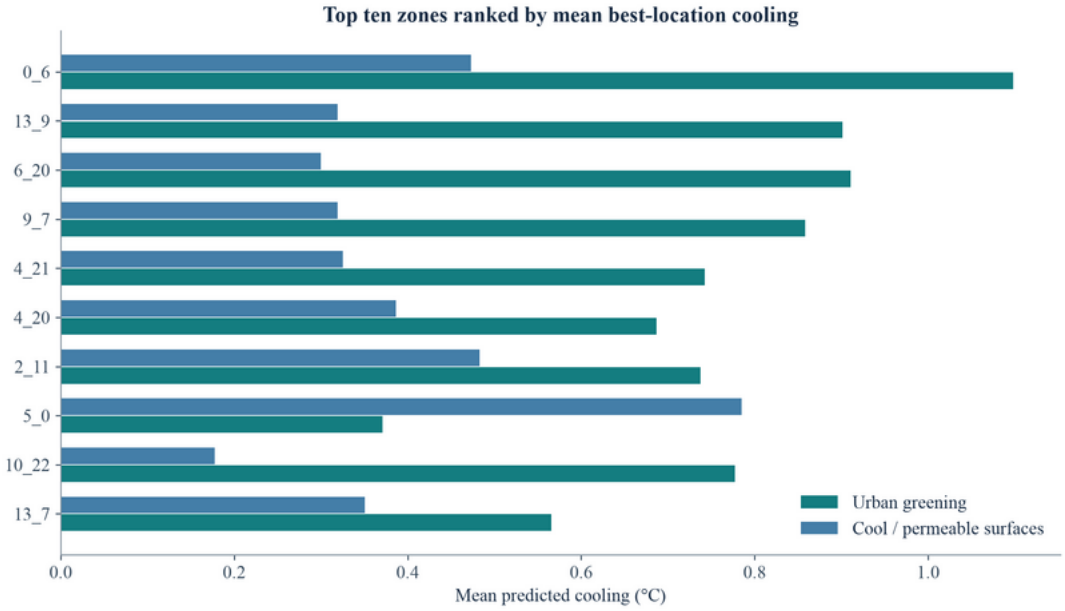


Figure 8.3: Top ten zones by mean of the best location-level cooling estimate

Zone 0_6 has the highest saved mean best-location cooling, approximately 1.170 °C. This differs from its mean greening-only effect of 1.098 °C because the best-location calculation can select different actions at different points. It should not be described as the effect of one uniform zone-wide intervention. The older README value of 0.84 °C belongs to an earlier output, not the final V3 table.

CHAPTER 9

TESTING AND VALIDATION

9.1 Testing Strategy and Data Integrity

Validation combined read-only repository checks, canonical inference tests, FastAPI request tests, local dashboard inspection, and an independent reproduction of the final spatial-CV design. The accompanying report/tools/validate_project.py script records 26 checks in validation_results.json. These checks establish implementation consistency within the supplied data; they are not a substitute for an independent field dataset or a formal security audit.

Table 9.1: Executed data and inference checks

Check group	Observed result
Raw V3 dimensions and completeness	Pass: 9,893 rows, 10 columns, no missing cells or duplicate rows
Feature schema and order	Pass: eleven inputs match the saved estimator and feature-order CSV
Land-cover encoding	Pass: all rows sum to one; coordinates excluded from predictors
Saved model versus stored outputs	Pass: maximum prediction difference below 0.00001 °C
HVI and Zone_ID lineage	Pass: equal-weight formula reproduced; 334 shared zones
Zero-intervention identity	Pass: all-zero controls produce exactly zero mean change
Maximum-control bounds	Pass: indices and normalised fractions remain valid at 100% settings
Canonical threshold boundaries	Pass: 30, 38, and 42 °C enter the expected higher categories
Notebook scenario reproduction	Pass: both multiplicative scenarios match stored predictions

The original project data and model were not modified. The validation script writes only report-support results. Source-code inspection was also used to identify semantic inconsistencies that a passing numerical test would not detect, including point-versus-zone labels and alternative validation definitions.

9.2 API, Integration, and Functional Testing

Table 9.2: Executed API and integration test cases

Test case	Expected behaviour	Result
GET /health	HTTP 200 and status ok	Pass
GET /model-info	HTTP 200 and eleven feature names	Pass
Valid /predict request	Prediction agrees with shared inference to 0.001 °C	Pass
Empty prediction list	HTTP 400	Pass
Incomplete feature record	HTTP 422	Pass
Control value above 100	HTTP 422	Pass
Control value below 0	HTTP 422	Pass
All-zero /simulate request	HTTP 200 and zero mean reduction	Pass
Default /simulate request	Matches canonical dashboard simulation after rounding	Pass
Local dashboard navigation	Seven sections inspected; charts and controls render in compatible environment	Pass with compatibility pin
CSV and analytical lineage	Prepared tables agree with their saved source products	Pass

The browser run used a local Streamlit server and a compatible Plotly version. The heat-driver, validation, and simulator views produced visible results. External basemap tiles could not be fetched in the restricted report environment, so static, data-derived geographic figures are used where a map illustration is needed. This is an environment limitation, not evidence that the hosted basemap is permanently unavailable.

9.3 Performance and Security Testing Boundaries

No concurrent-user load test, penetration test, formal accessibility audit, or long-duration reliability test was performed. Schema validation and bounded sliders reduce accidental invalid input, but they do not provide authentication or resource quotas. A production deployment should add these controls and measure latency and memory under representative workloads before making performance claims.

9.4 Independent Spatial-Validation Reproduction

The report reproduction used the saved estimator's parameter values, the model-ready V3 CSV, the notebook's 0.02-degree group construction, and five GroupKFold splits. It confirmed that no spatial group appeared in both training and test data within a fold. New fold models were fitted only for validation; the deployed pickle was left unchanged. Small numerical differences from the notebook are expected across library versions and execution environments.

Table 9.3: Saved notebook results and report reproduction

Metric	Saved mean fold result	Reproduced mean fold result	Pooled OOF result
R^2	0.5691	0.5684	0.5697
MAE (°C)	1.6645	1.6661	1.6661
RMSE (°C)	2.1163	2.1179	2.1181

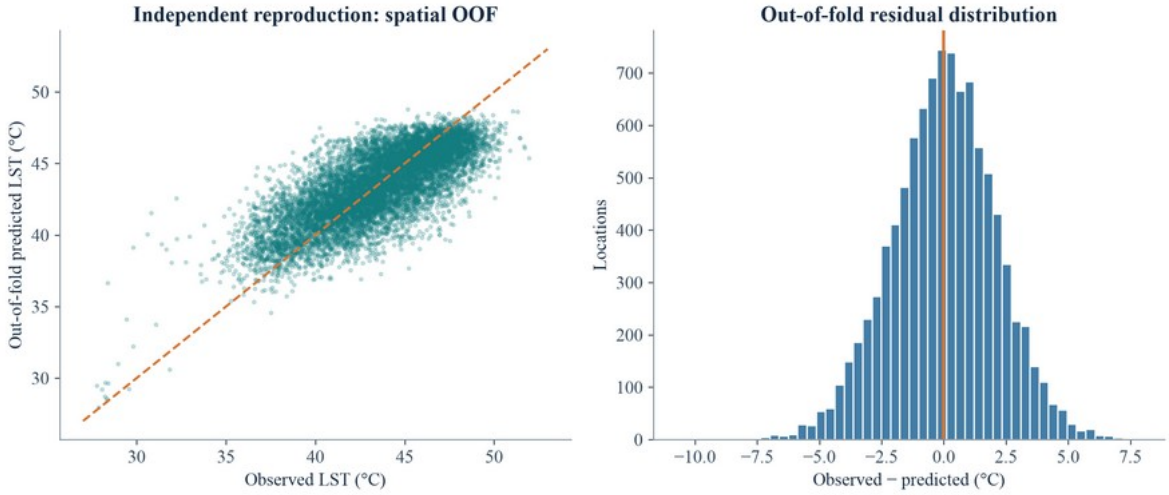


Figure 9.1: Reproduced spatial out-of-fold predictions and residuals

The close agreement supports the credibility of the saved V3 performance summary. It does not turn the final all-data model's fitted values into hold-out evidence. The residual spread and the moderate R^2 also show that substantial local variation remains unexplained. Predicted temperatures should be used with error awareness, especially near category thresholds.

CHAPTER 10

RESULTS AND DISCUSSION

10.1 Achievement of Project Objectives

Table 10.1: Objective-level discussion of the implemented outcome

Objective	Outcome and interpretation
Predict LST from physical and land-cover indicators	Implemented with an eleven-input XGBoost model; reproduced spatial MAE approximately 1.67 °C
Detect and prioritise heat hotspots	2,472 upper-quartile observed-LST points and ten saved priority zones; small-zone support requires caution
Explain heat drivers	NDBI leads the saved global SHAP ranking; correlated features and non-causal interpretation remain important
Account for population exposure	Equal-weight HVI produces a top-ten list distinct from the hottest-zone list
Compare cooling interventions	Two reproducible notebook alternatives plus a five-control live what-if simulator
Deliver decision-support access	Streamlit pages, standalone Folium maps, FastAPI endpoints, and CSV products

10.2 Scientific and Planning Interpretation

The results support three practical conclusions. First, heat mapping and vulnerability mapping answer different questions and should be inspected together. Second, model explainability can identify influential variables but cannot by itself establish the effect of constructing a park or replacing a roof. Third, location-specific scenario results can be negative, so intervention selection must include plausibility checks and field evidence.

The system’s value lies in connecting these questions within a traceable workflow. It provides a common spatial identifier, a consistent model, explicit assumptions, and exportable records. It does not eliminate the need for planners, climate specialists, engineers, or community consultation.

10.3 Limitations and Threats to Validity

1. Surface-temperature boundary: LST is not air temperature, thermal comfort, or a complete health-risk measure. Humidity, wind, exposure time, and personal vulnerability are not modelled.
2. Temporal provenance: acquisition dates, scene selection, cloud screening, and seasonal compositing cannot be reconstructed from the CSV alone. Transfer across seasons is untested.
3. Spatial validation: grouped folds reduce leakage, but adjacent blocks may remain correlated. Buffered or genuinely external spatial validation would provide stronger evidence.
4. Sampling and class imbalance: the sample is cropland-dominated, water has few records, and some high-ranked zones contain very few points. Area-weighted municipal estimates are not justified.
5. Exposure simplification: population is only one component of vulnerability. Age, income, housing quality, occupation, and cooling access are absent.
6. Scenario uncertainty: fixed sensitivity coefficients and multiplicative index changes are assumptions. Modified features can fall outside familiar physical combinations.
7. Optimisation boundary: the project compares scenarios but does not solve a budget-, land-, water-, or maintenance-constrained optimisation problem.
8. Reproducibility and operations: acquisition scripts and dependency pins are absent; older notebooks, helper defaults, labels, and API metadata are not fully aligned.

10.4 Responsible Use of the Results

Priority zones should be treated as candidates for investigation rather than final administrative decisions. A municipal workflow should verify surface conditions, check population and health context, compare engineering feasibility, estimate costs and water demand, and monitor outcomes after implementation. The system should not be used to deny services, infer individual health status, or justify demolition or land-use change without due process and local evidence.

CHAPTER 11

CONCLUSION AND FUTURE SCOPE

11.1 Advantages of the Developed System

UrbanHeat AI moves beyond a thermal map towards an integrated planning-support workflow. Its main advantage is traceability: data, feature definitions, trained artefacts, notebooks, reusable inference, and application outputs are connected in one repository. Separating offline preparation from interactive delivery makes the system understandable and extendable.

- Leakage-aware evaluation provides a more defensible performance estimate than an ungrouped random split.
- The final land-cover-aware model improves the saved V2 XGBoost comparison while retaining a compact feature schema.
- SHAP attribution and explicit validation metrics make model behaviour more transparent.
- Separate heat and vulnerability priorities prevent population exposure from being overlooked.
- Positive and negative scenario effects are preserved, supporting critical rather than promotional interpretation.
- A shared inference module keeps dashboard and API calculations consistent.
- CSV, HTML, and JSON outputs support planning review, research reproducibility, and technical integration.

11.2 Conclusion

The project achieves its core objective of detecting urban heat-stress patterns and organising cooling-strategy evidence for a sampled study area around Chhatrapati Sambhajinagar. The final V3 XGBoost model achieves a saved mean spatial-CV R^2 of 0.5691 and MAE of 1.6645 °C, with close agreement in an independent report reproduction. These results are useful but leave meaningful unexplained variation.

The strongest contribution is the integrated decision-support structure, not a claim of causal intervention optimisation. Hotspot analysis, explainability, vulnerability, scenario comparison, and accessible delivery are implemented. Responsible deployment requires better provenance, external validation, and calibrated intervention effects. Within those boundaries, the project supports further urban-climate research and planning.

11.3 Future Scope

Table 11.1: Proposed enhancements and their expected value

Enhancement	Purpose and required work
Recover and version acquisition workflows	Store Earth Engine scripts, scene dates, masks, sampling rules, and source-layer versions
Multi-season and multi-city validation	Test temporal transfer and geographic generalisation with independent hold-out regions
Higher-resolution thermal observations	Evaluate suitable airborne, local, or future thermal sources, including potential TRISHNA integration
National geospatial integration	Assess Bhuvan/NRSC land cover and MOSDAC weather context after licensing and compatibility review
Richer vulnerability indicators	Add age, housing, health proxies, cooling access, and socially appropriate deprivation measures
Field-calibrated intervention model	Measure before/after conditions and estimate uncertainty for trees, roofs, and permeable surfaces
Cost- and feasibility-aware optimisation	Add land availability, capital cost, maintenance, water demand, and equity constraints
Weather and air-temperature fusion	Combine satellite LST with station observations and humidity or radiation where available
Production hardening	Pin dependencies, add authentication and quotas, monitor artefact versions, and automate regression tests
Participatory planning interface	Support transparent review of proposed priorities with municipal teams and affected communities

11.4 Alignment with Earth-Observation Applications

The framework is aligned with the broader use of Indian and international Earth observation for urban resilience. Bhuvan, NRSC, NICES, MOSDAC, and TRISHNA are potential future integrations, not completed data sources in this repository. Their inclusion should follow documented access, validation, and suitability checks. The present project demonstrates the analytical and application structure into which such datasets could later be incorporated.

APPENDIX A: FEASIBILITY AND COMPLEXITY ASSESSMENT

A.1 Feasibility Summary

The current file-based system is technically feasible on a commodity CPU because the final dataset contains fewer than ten thousand rows and eleven predictors. Open-source software reduces licensing barriers. The main operational costs are reliable data preparation, expert interpretation, field verification, hosting, and maintenance. Schedule feasibility depends on preserving the dependency order between acquisition, validation, analysis, and application integration.

A.2 Computational Complexity

Table A.1: Indicative complexity of major operations

Operation	Indicative cost	Explanation
Audit / descriptive statistics	O(nf)	Scan n records and f fields
One-hot land-cover encoding	O(nc)	Encode c land-cover classes
Grouped five-fold validation	O(5 × T_train)	Repeated model fitting dominates
XGBoost batch prediction	Approximately O(nTd)	n rows traversing T trees of depth-related d; implementation-dependent
SHAP tree attribution	Depends on sample, trees, depth, and features	Notebook limits explanation to 1,000 rows
Live scenario preparation	O(nf)	Copy, modify, clip, and normalise features
Zone aggregation	O(n) expected with hash grouping	Group records by Zone_ID
Interactive map preparation	O(min(n, 2000)) plus rendering	Fixed display sample in current dashboard
CSV export	O(nf_out)	Serialise output fields

Here n is the number of locations, f the number of fields, c the number of classes, T the number of trees, and d a depth-related traversal factor. These are planning-level expressions, not measured latency guarantees. Actual performance depends on library versions, hardware, thread count, and data characteristics.

APPENDIX B: PAPER PUBLICATION / PRESENTATION

B.1 Publication and Presentation Record

The supplied repository identifies development for Bharatiya Antariksh Hackathon 2026, Problem Statement PS-01. It does not include an accepted research paper, conference presentation certificate, journal volume, DOI, or publication licence. No paper publication or formal presentation outcome is claimed in this report.

This section follows the structure of the supplied reference report and is reserved for verified publication or presentation details. The project team should insert only documents that correspond to this project and have actually been accepted, submitted, or presented, with the status stated accurately.

Table B.1: Publication details to be completed when applicable

Field	Entry
Title of paper	[Enter the verified project paper title]
Authors	[Enter the author list in publication order]
Venue / journal	[Enter the conference or journal name]
Status	[Prepared / Submitted / Accepted / Presented / Published]
Date and identifier	[Enter date, DOI, ISBN, ISSN, or proceedings details if available]
Supporting evidence	[Attach acceptance letter, certificate, or camera-ready first page]

B.2 Repository and Demonstration Record

Primary implementation: omkarrr2533/urban-heat-stress-hotspot-detection. The inspected base revision is 56ff9b0857eb35d773c3e1cedc0aa3c4e1ec4376. The repository README credits Team AstroByte and links a Streamlit deployment. Local dashboard captures in this report come from the checked-in application, not from an invented interface. A formal publication record remains separate from these repository and demonstration artefacts.

APPENDIX C: PLAGIARISM REPORT

C.1 Institutional Similarity-Check Record

This page is reserved for the plagiarism or similarity-check report required by the department. No institutional plagiarism scan was supplied with the project repository or reference-format document, and no similarity percentage or clearance certificate is fabricated here. Before final submission, the completed report should be checked using the institute's designated tool and the authorised result attached.

Table C.1: Details to be completed after the institutional check

Field	Entry
Student / project details	[Enter final submission details]
Checking tool	[Enter the institute-approved service]
Check date	[Enter date]
Report reference number	[Enter the generated verification identifier]
Similarity result	[Enter the tool-generated result without alteration]
Reviewer / department approval	[Enter authorised review details and signature]

C.2 Source Attribution and Academic Responsibility

Project-specific descriptions, feature definitions, implementation details, and numerical results in this report are derived from the supplied repository and its saved outputs. General methodological context is attributed in the reference list. The reference report was used to guide the academic sequence and presentation, not as a source of unrelated project results. Directly reused third-party material should remain quoted and cited, and the student team is responsible for verifying the final document against institutional academic-integrity rules.

Attach the authorised similarity report after this page or replace this placeholder page according to the department's final submission instructions. Completion of the student names, examination numbers, signatures, publication record, and plagiarism check is required before the document is treated as an approved institutional submission.

REFERENCES

[1] T. R. Oke, “The energetic basis of the urban heat island,” Quarterly Journal of the Royal Meteorological Society, vol. 108, no. 455, pp. 1–24, 1982. doi: 10.1002/qj.49710845502.

[2] Q. Weng, “Thermal infrared remote sensing for urban climate and environmental studies: Methods, applications, and trends,” ISPRS Journal of Photogrammetry and Remote Sensing, vol. 64, no. 4, pp. 335–344, 2009.

[3] U.S. Geological Survey, “Landsat Collection 2 Level-2 Surface Temperature Product.” USGS Landsat Missions. Available: https://www.usgs.gov/landsat-missions/landsat-collection-2-surface-temperature.

[4] European Space Agency, “Sentinel-2 Mission and Surface Reflectance Products.” Sentinel Online. Available: https://sentinels.copernicus.eu/web/sentinel/missions/sentinel-2.

[5] N. Gorelick et al., “Google Earth Engine: Planetary-scale geospatial analysis for everyone,” Remote Sensing of Environment, vol. 202, pp. 18–27, 2017. doi: 10.1016/j.rse.2017.06.031.

[6] T. G. Farr et al., “The Shuttle Radar Topography Mission,” Reviews of Geophysics, vol. 45, RG2004, 2007. doi: 10.1029/2005RG000183.

[7] A. J. Tatem, “WorldPop, open data for spatial demography,” Scientific Data, vol. 4, 170004, 2017. doi: 10.1038/sdata.2017.4.

[8] D. Zanaga et al., “ESA WorldCover 10 m 2020 v100,” European Space Agency, 2021. doi: 10.5281/zenodo.5571936.

[9] L. Breiman, “Random Forests,” Machine Learning, vol. 45, pp. 5–32, 2001. doi: 10.1023/A:1010933404324.

[10] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785–794, 2016. doi: 10.1145/2939672.2939785.

[11] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” Advances in Neural Information Processing Systems, vol. 30, 2017.

[12] D. R. Roberts et al., “Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure,” Ecography, vol. 40, pp. 913–929, 2017. doi: 10.1111/ecog.02881.

[13] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[14] Streamlit, “Streamlit Documentation.” Available: https://docs.streamlit.io/. FastAPI, “FastAPI Documentation.” Available: https://fastapi.tiangolo.com/.

[15] Plotly, “Plotly Python Graphing Library.” Available: https://plotly.com/python/. Python-Visualization, “Folium Documentation.” Available: https://python-visualization.github.io/folium/.

[16] UrbanHeat AI project repository, Team AstroByte, “Urban Heat Mitigation and Cooling Strategy Optimization,” supplied source code, notebooks, datasets, and generated outputs, inspected revision 56ff9b0, 2026.